

Solución a Punto 3 de preguntas teóricas Prueba Técnica – Desarrollador Java Spring Boot + JavaScript

1. Diferencias entre @Controller, @RestController y @Service

@Controller

- Se usa para aplicaciones web tradicionales (MVC).
- Retorna **vistas** (HTML, JSP, Thymeleaf).
- Los métodos devuelven Strings que representan nombres de plantillas.

@RestController

- Es una combinación de @Controller + @ResponseBody.
- Se usa para **APIs REST**.
- Los métodos retornan directamente **JSON o XML**.
- No devuelve vistas.

@Service

- Indica una **capa de negocio**.
- Alberga la lógica central del sistema.
- Separa responsabilidades del controlador.
- Facilita pruebas unitarias y mantenimiento.

2. Qué es un DTO y uso recomendado

Un **DTO (Data Transfer Object)** es un objeto diseñado para transportar datos entre capas sin exponer directamente las entidades del sistema.

Uso recomendado

- Cuando no queremos exponer toda la información de la entidad.
- Para validar datos de entrada sin afectar las clases del modelo.
- Para estructurar mejor las respuestas que ve el cliente.
- Para evitar problemas de seguridad, acoplamiento o sobreexposición de datos.

Ejemplo:

Un `ProductoEntity` puede tener campos internos que no deben enviarse al cliente
→ el `DTO` solo contiene los necesarios.

3. Explicar Promesa en JavaScript

Una **Promesa** es un objeto que representa una operación asíncrona y su resultado futuro.

Permite ejecutar código sin bloquear el hilo principal.

Tiene 3 estados:

- **pending** (pendiente)
- **fulfilled** (resuelta)
- **rejected** (rechazada)

Se consume con:

`promise.then().catch();`

O con:

`async/await`

¿Para qué sirve?

- Llamadas a APIs
- Operaciones que tardan (timeouts, lectura de archivos, consultas remotas)
- Evita el “callback hell” y organiza el flujo asíncrono

4. Ventaja de usar H2 en pruebas locales

- Es una **base de datos en memoria**, ligera y muy rápida.
- No requiere instalación adicional.
- Ideal para pruebas y desarrollo rápido.
- Se reinicia cada vez que corre la aplicación (ambientes limpios).
- Permite inspeccionar datos desde la consola H2.

Por eso se usa para desarrollar y probar, pero no en producción.

5. Qué mejorarías para un entorno productivo

Principales mejoras recomendadas:

✓ Seguridad

- Activar autenticación JWT o OAuth2
- Configurar CORS correctamente
- Validar entradas y sanitizar datos

✓ Base de datos

- Usar PostgreSQL con migraciones (Flyway o Liquibase)
- Conexiones pool (HikariCP)
- Backups automáticos

✓ Documentación

- Añadir Swagger/OpenAPI

✓ Logs

- Configurar logs estructurados (JSON)
- Integración con herramientas como ELK

✓ Manejo de errores

- Respuestas personalizadas y consistentes
- Métricas y monitoreo (Micrometer + Prometheus)

✓ Despliegue

- Docker + Docker Compose
- Pipelines CI/CD
- Ambiente separado: dev, qa, prod

✓ Frontend

- Hospedar en un servidor estático (Nginx / S3)
- Usar HTTPS

